Singleton

Singleton Pattern

One instance, one front door

DEFINITION

Guarantee a class has exactly one instance per process and a single global access point.

Singleton fixes how many of a thing exist at exactly one and publishes a single global access point to reach it. The intent is control, not convenience: the class itself owns its one instance and the path to get it, instead of letting callers new their own copies. Modern caveat — the same one-instance guarantee usually belongs in a DI container's singleton scope, not a hand-rolled global.

WHY IT MATTERS

A second, accidental instance breaks every assumption built on "there is only one" — two caches, two config copies, two pools drifting out of sync. But the global access point is the catch: any module can reach the singleton without declaring it, so dependencies go invisible, tests can't substitute a fake, and init-order bugs hide in plain sight. You buy one guaranteed identity and pay in hidden coupling.

How it works

Private ctor	Blocks new from outside, so no other code can mint a second instance — every creation path funnels inward.
Static field	One cached slot that holds the sole instance for the whole process.
getInstance()	The single global door — lazily builds on the first call, then returns the cached one.

HOW TO APPLY

1. Confirm you truly need one instance, not just convenient global access.
2. Make creation unreachable from outside (a private ctor or a module-private factory).
3. Cache the instance in one static slot; build it lazily on first request.
4. Expose one accessor — or better, register it singleton-scoped in your DI container.

> LITMUS TEST
>
> Would a second instance be a bug, not just waste? If two copies merely cost memory, you want caching. Singleton is for when duplicate identity breaks correctness.

— Smell → fix

Every caller news its own Config, so "shared" settings quietly diverge:

```
// every caller can build its own Config
class Config { env = loadEnv() }
const a = new Config(), b = new Config()
// a !== b — two configs drift out of sync
```

↓ FUNNEL CREATION THROUGH ONE DOOR

```
class Config {
static #i: Config // the one cached object
static get(): Config { // single global door
return (Config.#i ??= new Config())
}
}
// Node idiom: export const config = new Config()
```

Creation now funnels through Config.get(), which caches the one instance; lock the ctor to private so new can't escape the class. In Node a cached module export gives the same guarantee — and a DI singleton scope gives it without the hidden global.

USE WHEN

- ✓ One genuinely shared resource (config, connection pool)
- ✓ A single well-known access point is required

AVOID WHEN

- ▲ Most cases — prefer a DI singleton-scoped service

— Trade-offs & pitfalls

PROS

+ Controlled single access point

CONS

− Hidden global state

− Hard to mock

− Init-order and test coupling

COMMON PITFALLS

▲ Using Singleton as a socket for global variables — convenient reach now, hidden coupling and untestable code later.

▲ Assuming "one per process": worker_threads, serverless warm starts, and duplicate bundled copies each mint their own — it is one-per-realm, not truly global.

▲ Naive lazy getInstance() is not thread-safe outside single-threaded Node; under real threads you need eager init, a lock, the holder idiom, or an enum.

WHERE YOU SEE IT

- A NestJS @Injectable() provider in default singleton scope — one instance the container owns and can inject (and swap in tests).
- java.lang.Runtime.getRuntime() and Spring's default singleton-scoped beans.
- A Node module that exports new Logger() — the module cache makes it a de-facto singleton.

SEE ALSO

Dependency Injection	the modern replacement — same one-instance guarantee, but injectable and mockable
Factory Method	getInstance() is a static creator that hides which concrete instance you get
Abstract Factory	a concrete factory is often itself kept as a single shared instance (GoF)
Facade	a single front-door object is frequently kept as one Singleton

Interview & sources

Why is Singleton often called an anti-pattern?

It is hidden global state: tight coupling, hard to test, and an unclear lifecycle. Most cases are better served by a DI singleton-scoped service.

Is getInstance() thread-safe by default?

No — naive lazy creation isn't. Use eager init, double-checked locking, the holder idiom, or an enum (Effective Java).

Singleton vs a static class?

A Singleton is a real object: it can implement interfaces, be subclassed, mocked, and passed around. A static utility class can't.

What bites in serverless?

Instance state survives across warm invocations, so it can leak or surprise between requests — keep singletons stateless or scope per request.

REMEMBER

One instance per process, reached through one well-known door — but that door is hidden global state.

SOURCES

GoF — "Design Patterns: Elements of Reusable OO Software" (1994): Singleton (creational)

Joshua Bloch — "Effective Java" (3rd ed., 2018): Item 3 — enforce the singleton property; prefer dependency injection